TASK: 8

XSS & HTMLI

Surendra Gabbani

HTMLI

What is HTML Injection?

HTML Injection is a type of web security vulnerability where an attacker is able to inject raw HTML code into a web page due to insufficient input validation or sanitization. Unlike XSS (Cross-Site Scripting), which often targets the execution of JavaScript, HTML Injection primarily alters the structure and content of the rendered page.

How It Works

When a web application takes user-supplied input and reflects it back into the HTML response without proper encoding or filtering, attackers can craft malicious input such as:

```
<b>Injected Bold Text</b>
<h1>Fake Heading</h1>
<img src="fake.png" onerror="alert('HTMLi')">
```

Types of HTML Injection

Reflected HTMLi

- Input immediately reflected in server response (e.g., search, error messages).

Stored HTMLi

- Malicious HTML stored on server (e.g., comments, user profiles) and served to other users.

How HTMLi can lead to XSS

- HTML injection may allow insertion of attributes like onerror in <img> or href="javascript:..." if not filtered.
- Poor sanitizers that remove <script> but allow event handlers or style attributes can be bypassed.

XSS

What is XSS?

Cross-Site Scripting (XSS) is a web application vulnerability that allows attackers to inject malicious scripts (mostly JavaScript) into websites. When unsuspecting users visit the page, their browser executes the attacker's code, thinking it's legitimate.

How XSS Works:

1. An attacker finds a way to inject malicious code (like <script>alert('Hacked!')</script>) into a web page.
2. When another user visits the page, the browser executes that malicious script as if it was trusted.
3. This can lead to stealing cookies, session tokens, or sensitive data, and even full account takeover.

Types of XSS:

Stored XSS (Persistent):

- Malicious script is permanently stored on the server (e.g., in a comment section, forum, profile bio).
- Every time a victim visits, the script runs.

Reflected XSS (Non-persistent):

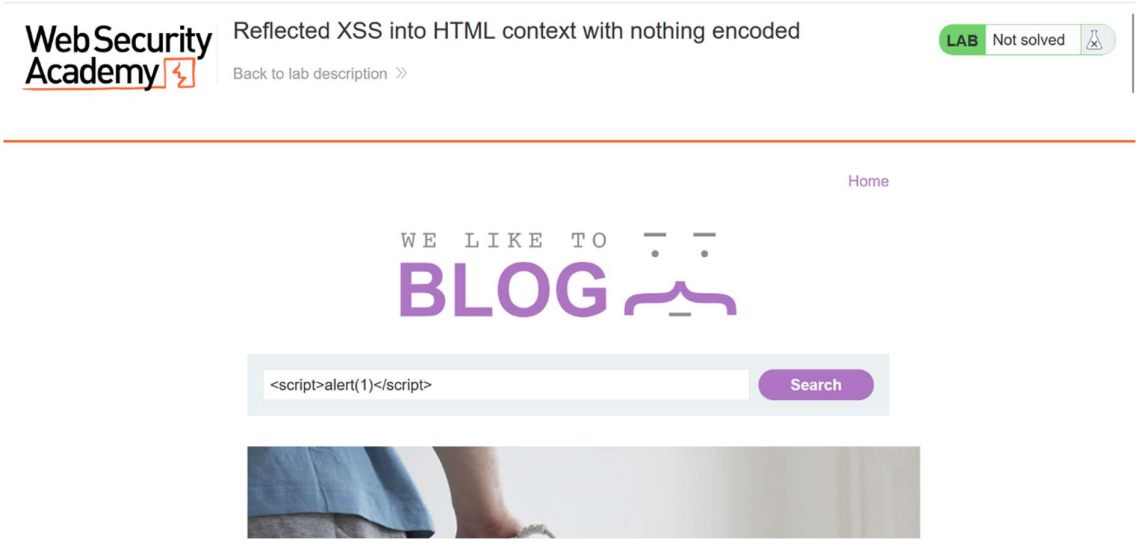
- Malicious code comes from a request (e.g., URL or search parameter) and is reflected back in the response.
- Common in GET/POST parameters.
    -

DOM-based XSS:

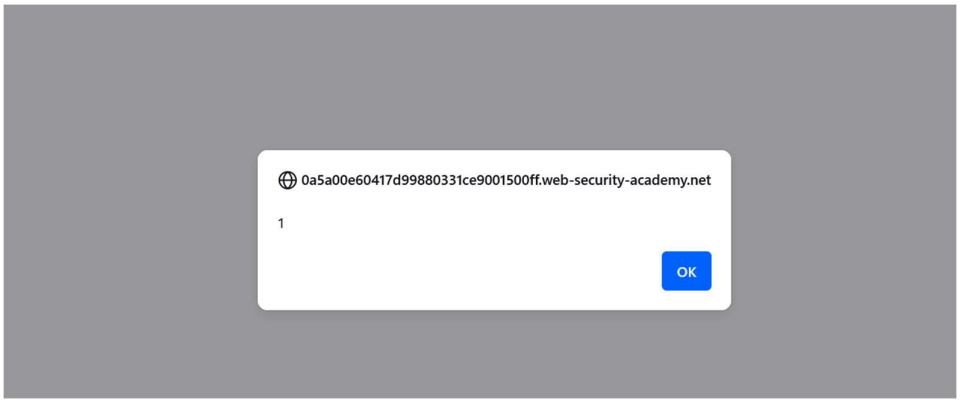
- Happens entirely on the client side, where JavaScript modifies the DOM insecurely.
- Example: document.location.hash being directly written to the page without sanitization.

LAB -1(HTMLI)

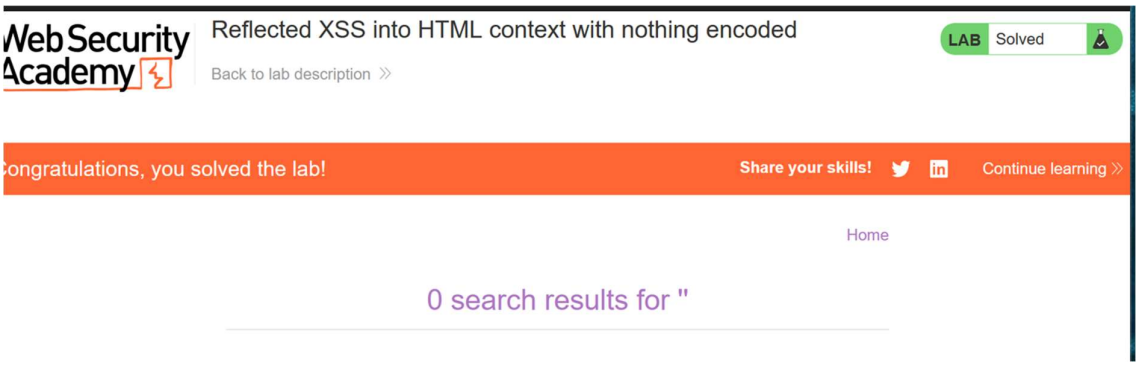
STEP-1



STEP2:



STEP 3: solved



LAB -2(XSS)

Lab: DOM XSS in `document.write` sink using source `location.search`

APPRENTICE

LAB Not solved

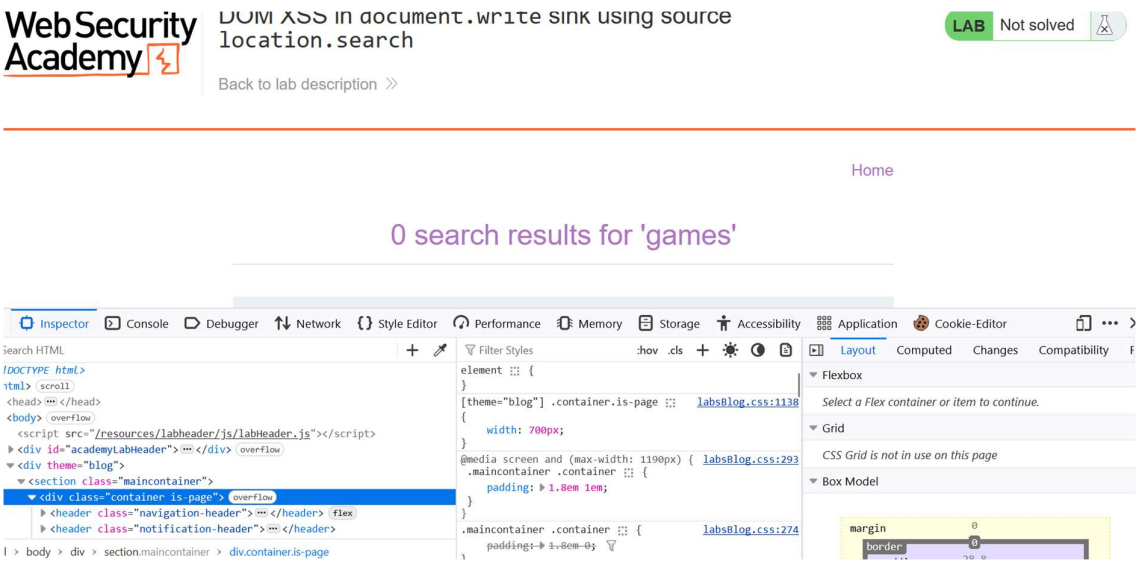
This lab contains a DOM-based cross-site scripting vulnerability in the search query tracking functionality. It uses the JavaScript `document.write` function, which writes data out to the page. The `document.write` function is called with data from `location.search`, which you can control using the website URL.

To solve this lab, perform a cross-site scripting attack that calls the `alert` function.

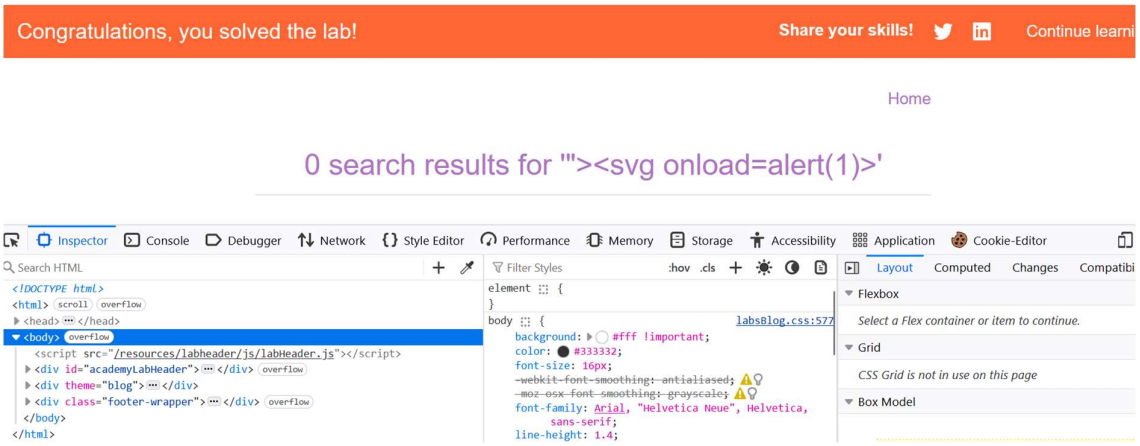
ACCESS THE LAB

STEP1:

(Search anything in search bar and click inspect)



STEP2:(Inject java script in the website) Solved



XSS Can Be Used For:

- Stealing cookies/session tokens
- Keylogging
- Redirecting users to phishing pages
- Defacing websites
- Performing actions as the victim

Prevention:

- Escape output (HTML encode special characters).
- Use input validation & sanitization.
- Content Security Policy (CSP) to limit script execution.
- HTTPOnly cookies to protect session tokens.
- Avoid using innerHTML with untrusted data.

Conclusion – XSS (Cross-Site Scripting):

Cross-Site Scripting (XSS) is one of the most common and dangerous web vulnerabilities because it allows attackers to execute malicious scripts directly in a user's browser. It can lead to data theft, account compromise, and severe security breaches. Preventing XSS requires proper input validation, output encoding, secure cookie handling, and strong browser-side protections like CSP. By following secure coding practices, developers can significantly reduce the risk of XSS attacks and protect both applications and users.

THANK YOU